



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 5. Работа со строками и регулярными выражениями

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

- 📶 Строки в Python
- 📶 Запись строк
- 📶 Специальные символы
- 📶 Простейшие операции
- 📶 Методы строк
- 📶 Встроенные строковые функции
- 📶 Форматирование строк
- 📶 Регулярные выражения
- 📶 Шаблоны
 - 🕒 Базовые шаблоны
 - 🕒 Сложные шаблоны
- 📶 Методы регулярных выражений
- 📶 Группы

Строки в Python (1 из 3)

- 📡 Строка – последовательность символов, заключённая в кавычки
- 📡 Переменная строкового типа предназначена для хранения и представления текстовой информации
- 📡 Любая строка является объектом класса **str**
 - 🕒 Это даёт возможность использовать методы класса **str**

```
string_1 = 'Hello'  
string_2 = "world!"
```

```
print(string_1, string_2)
```

Вывод: Hello world!

Строки в Python (2 из 3)

☞ Строка состоит из индексированной последовательности символов

- ☉ Каждый символ имеет свой индекс

☞ Положительные индексы:

- ☉ Индекс первого символа строки: 0
- ☉ Индекс последнего: Длина строки – 1

☞ Отрицательные индексы:

- ☉ Индекс последнего символа строки: -1
- ☉ Индекс первого: -Длина строки

-6	-5	-4	-3	-2	-1
S	T	R	I	N	G
0	1	2	3	4	5

Строки в Python (3 из 3)

- Для доступа к символу используется конструкция:
имя_строки[индекс]
 - Имя строки – имя переменной, хранящей строку
 - Индекс – индекс символа
- Попытка указать не существующий в данной строке индекс приведет к ошибке `IndexError`

```
str_1 = "string"
```

```
print(str_1[0]) Вывод: "s"
```

```
print(str_1[-1]) Вывод: "g"
```

```
print(str_1[20]) Вывод: IndexError: string index out of range
```

Запись строк (1 из 2)

- Для записи строки используются одинарные, двойные и тройные кавычки

```
string_1 = 'Hello'  
string_2 = "world!"
```

- Если внутри строки нужны кавычки, то нужно использовать другой вид кавычек

```
string_1 = "Строка с 'кавычками' внутри"
```

Вывод: Строка с 'кавычками' внутри

- Для записи многострочного блока текста используются тройные кавычки

```
string_1 = '''Строка 1  
Строка 2'''
```

Запись строк (2 из 2)



```
str_1 = "Hello world"  
str_2 = 'Hello world'  
str_3 = "Word: 'hello'"  
str_4 = '''Многострочный  
текст с 'кавычками'  
внутри'''
```



```
str_1 = "Hello world"  
str_2 = "Word: "hello""
```

Специальные символы (1 из 5)

- 🔊 Для записи специальных символов используется экранирование
- 🔊 Символ экранирования – обратный слеш (\) указывает на то, что один или несколько последующих символов нужно интерпретировать особым образом
- 🔊 Экранированная последовательность в среде разработки выделяется цветом
- 🔊 Для отмены экранирования символов в строке перед строкой необходимо поставить символ r

```
str_1 = "Строка в двойных кавычках содержит двойные \" кавычки"
```

Вывод: Строка в двойных кавычках содержит двойные " кавычки

```
str_1 = r"Строка в двойных кавычках содержит двойные \" кавычки"
```

Вывод: Строка в двойных кавычках содержит двойные \ " кавычки

Специальные символы (2 из 5)

 \newline – запись многострочного текста

```
str_1 = 'Это очень \
длинная строка'
```

Вывод: Это очень длинная строка

📶 \ - запись обратного следа

 \ ' – запись одинарной кавычки (апострофа)

 \ “ – запись двойных кавычек

```
str_1 = "Это строка с слешем \\", одинарной кавычкой \' и двойными  
кавычками \'"
```

Вывод: Это строка с слешем \, одинарной кавычкой ' и двойными кавычками "

Специальные символы (3 из 5)

Перемещение курсора:

- ⦿ `\n` – переход на новую строку
- ⦿ `\b` – удаление одного символа (аналог backspace)
- ⦿ `\r` – возврат курсора в начало строки
- ⦿ `\f` – разрыв страницы

```
str_1 = "Строка 1\nСтрока 2"
```

Вывод: Строка 1
 Строка 2

```
str_2 = "Удаление одного символа\b"
```

Вывод: Удаление одного символ

```
str_3 = "Это будет перезаписано\rНовое содержимое строки"
```

Вывод: Новое содержимое строки

Специальные символы (4 из 5)

📶 Виды табуляции:

- ⦿ \t – горизонтальная табуляция
- ⦿ \v – вертикальная табуляция

📶 \a – звуковой сигнал

```
str_1 = "\tГоризонтальный отступ"  
str_2 = "\vВертикальный отступ"  
str_3 = "\t\vКомбинированный отступ\a"
```

Звуковой сигнал

Вывод:

Горизонтальный отступ

Вертикальный отступ

Комбинированный отступ

Специальные символы (5 из 5)

- Можно вставлять символы используя:
 - `\0oo` – восьмеричный код символа (две восьмеричные цифры после `\0`)
 - `\xhh` – шестнадцатеричный код символа (две шестнадцатеричные цифры после `\x`)
 - `\N{id}` – имя символа в Unicode
- Коды символов можно получить из таблиц Unicode и ASCII

```
str_1 = "Строка с восьмеричным кодом\041"  
str_2 = "Строка с шестнадцатеричным кодом\x21"  
str_3 = "Строка со смайликом \N{grinning face}"
```

Вывод:

Строка с восьмеричным кодом!

Строка с шестнадцатеричным кодом!

Строка со смайликом 😊

Простейшие операции (1 из 2)

- 🔊 Строка – это неизменяемый тип данных
 - 🔊 Результатом выполнения всех операций будет новый объект, содержащий новую строку
- 🔊 Конкатенация (сложение строк)
 - 🔊 `str_1 + str_2`
- 🔊 Повторение строки
 - 🔊 `str_1 * 2`

```
str_1 = "Hello"
str_2 = "world "
str_3 = str_1 + " " + str_2
Вывод: Hello world
str_4 = str_3 * 3
Вывод: Hello world Hello world
```

Простейшие операции (2 из 2)

📡 Извлечение среза (slice) – получение подстроки

📡 Существует 3 формы:

- 🕒 `str[i]` – получение одного символа
- 🕒 `str[a:b]` – получение подстроки, состоящей из символов с индексами от **a** включая, до **b** не включая
- 🕒 `str[a:b:step]` – получение подстроки, состоящей из каждых **step** символов из диапазона от **a** до **b**

```
str_1 = "Hello world!"
```

```
print(str_1[2]) Вывод: "l"
```

```
print(str_1[0:4]) Вывод: "Hell"
```

```
print(str_1[0:10:3]) Вывод: "Hlwl"
```

Методы строк (1 из 4)

Регистр строки:

- `string.lower()` – перевод строки в нижний регистр
- `string.upper()` – перевод строки в верхний регистр
- `string.capitalize()` – переводит первый символ в верхний регистр, остальные в нижний
- `string.swapcase()` – меняет регистр каждого символа на противоположный
- `string.title()` – делает первые буквы слов заглавными

<code>str_1 = "Hello world!"</code>	
<code>print(str_1.lower())</code>	Вывод: "hello world!"
<code>print(str_1.upper())</code>	Вывод: "HELLO WORLD!"
<code>print(str_1.capitalize())</code>	Вывод: "Hello world!"
<code>print(str_1.swapcase())</code>	Вывод: "hELLO WORLD!"
<code>print(str_1.title())</code>	Вывод: "Hello World!"

Методы строк (2 из 4)

Поиск подстроки в строке:

- `string.find(sub[, <start>[, <end>]])` – возвращает индекс первого вхождения подстроки **sub** в строку **string**. Если вхождений нет, то возвращается -1
- `string.count(sub[, <start>[, <end>]])` – возвращает количество вхождений подстроки **sub** в строку **string**
- `string.index(sub[, <start>[, <end>]])` – аналог `string.find()`, но, если вхождений нет, то вызывается исключение `ValueError`
- `string.replace(old, new[, count])` – заменяет вхождения подстроки **old** на подстроку **new**. Необязательный параметр **count** – количество замен

Необязательные параметры `start` и `end` задают отрезок строки, в пределах которого происходит поиск

```
str_1 = "Hello world!"  
print(str_1.find("ll"))  Вывод: 2  
print(str_1.count("l"))  Вывод: 3  
print(str_1.index("Hi")) Вывод: ValueError: substring not found  
print(str_1.replace("l", "1", 2)) Вывод: "He11o world!"
```

Методы строк (3 из 4)

Проверка строк:

- `string.isdigit()` – возвращает `True`, если строка состоит только из цифр
- `string.isalpha()` – возвращает `True`, если строка состоит только из букв
- `string.islower()` – возвращает `True`, если строка в нижнем регистре
- `string.isupper()` – возвращает `True`, если строка в верхнем регистре
- `sub in string` – возвращает `True`, если подстрока `sub` присутствует в строке `string`

```
str_1 = "hello world!"
```

<code>print(str_1.isdigit())</code>	Вывод: <code>False</code>
<code>print(str_1.isalpha())</code>	Вывод: <code>False</code>
<code>print(str_1.islower())</code>	Вывод: <code>True</code>
<code>print(str_1.isupper())</code>	Вывод: <code>False</code>
<code>print("ll" in str_1)</code>	Вывод: <code>True</code>

Методы строк (4 из 4)

Разделение и объединение строк:

- ⦿ `string.split(sep=None, maxsplit=-1)` – деление строки на список из подстрок. Если сепаратор (`sep`) не указан, то строка делится по символам пробелов. Параметр `maxsplit` позволяет указать число делений
- ⦿ `string.join(iter)` – сложение элементов объекта `iter` в строку, используя в качестве разделителя `string`. Все элементы объекта `iter` должны быть строками

```
str_1 = "Hello world!"
```

```
print(str_1.split())
```

Вывод: ['Hello', 'world!']

```
list_1 = ["L", "i", "s", "t"]
```

```
print(("").join(list_1))
```

Вывод: "List"

Встроенные строковые функции (1 из 2)

 Наиболее популярные функции:

- ⦿ `chr(code)` – преобразование кода ASCII (`code`) в символ
- ⦿ `ord(symbol)` – преобразование символа (`symbol`) в код ASCII
- ⦿ `len(string)` – возвращает длину строки (количество символов)
- ⦿ `str(obj)` – возвращает строковое представление объекта

```
str_1 = "Hello world" + chr(33)
```

```
print(str_1)                                Вывод: Hello world!
```

```
print(len(str_1))                           Вывод: 12
```

```
print(ord("!"))                             Вывод: 33
```

```
print(str(print))                           Вывод: <built-in function print>
```

Встроенные строковые функции (2 из 2)

📡 Функции преобразования строк в различные структуры:

- ◉ `int(string)` – преобразование строки в число целого типа
- ◉ `float(string)` – преобразование строки вещественное число
- ◉ `list(string)` – создание списка из символов строки
- ◉ `tuple(string)` – создание кортежа из символов строки
- ◉ `set(string)` – создание набора уникальных символов строки

📡 Если переданная в методы `int()` и `float()` строка не является числом, то возникнет ошибка

```
print(int("3"))
```

Вывод: 3

```
print(float("2.7"))
```

Вывод: 2.7

```
print(list("Hello"))
```

Вывод: ['H', 'e', 'l', 'l', 'o']

```
print(tuple("Hello"))
```

Вывод: ('H', 'e', 'l', 'l', 'o')

```
print(set("Hello"))
```

Вывод: {'o', 'H', 'e', 'l'}

Форматирование строк (1 из 6)

Будут рассмотрены 2 способа форматирования:

- С использованием метода `.format()`
- С использованием f-строк. Версия Python ≥ 3.6

Пример простейшего форматирования:

```
date = 15  
print("Today is", date, "th January")
```

Громоздко и
неудобно

Пример продвинутого форматирования:

```
date = 15  
print(f"Today is {date}th January")
```

Гораздо лучше

Форматирование позволяет гораздо проще и эффективнее работать со строками

Форматирование строк (2 из 6)

🌀 В базовом варианте для форматирования строки при помощи метода `.format()` необходимо:

1. В строке указать поля замены при помощи скобок `{ }`
2. У строки вызвать метод `.format()`
3. Передать в метод подставляемые значения

`"Hello {}!".format("world")` **Вывод:** `"Hello world!"`

🌀 Если полей замены несколько, то значения подставляются по порядку, согласно нумерации или по имени

`"{ } {}".format("Hello", "world")` **Вывод:** `"Hello world!"`

`"{1} {0}!".format("Hello", "world")` **Вывод:** `"world Hello!"`

`"Hello {obj}!".format(obj = "world")` **Вывод:** `"Hello world!"`

Форматирование строк (3 из 6)

- Подставляемые в поля замены значения можно форматировать
- Схема форматирования
 - "{" [имя поля или номер] ["!" преобразование] [":" спецификация] "}"
- В спецификации может указываться:
 - Заполнитель пробелов
 - Выравнивание
 - Знак и точность, если подставляется число
 - Ширина поля замены
 - Тип подставляемого значения

```
data = (3.1415, "cm")
```

```
"Pi = {0[0]:.2d} {0[1]:s}".format(data)
```

Вывод: Pi = 3.14 cm

Форматирование строк (4 из 6)

 Схема спецификации:

- ⦿ [[fill] align] [sign] [#] [0] [width] [,] [.precision] [type]

 Возможные значения:

- ⦿ fill – любой символ
- ⦿ align – "<", ">", "^", "="
- ⦿ sign – "+", "-", " "
- ⦿ width – целое число
- ⦿ precision – целое число
- ⦿ type – "b", "c", "d", "e", "E", "f", "F", "g", "G", "n", "o", "s", "x", "X", "%"

```
print("{:*^30.2f}".format(12.345))
```

Вывод: *****12.35*****

Форматирование строк (5 из 6)

📡 Схема спецификации:

⦿ [[fill] align] [sign] [#] [0] [width] [,] [.precision] [type]

📡 Возможные значения:

- ⦿ fill – любой символ
- ⦿ align – "<", ">", "^", "="
- ⦿ sign – "+", "-", " "
- ⦿ width – целое число
- ⦿ precision – целое число
- ⦿ type – "b", "c", "d", "e", "E", "f", "F", "g", "G", "n", "o", "s", "x", "X", "%"

Заполнитель



```
print("{:*^30.2f}".format(12.345))
```

Вывод: *****12.35*****

Форматирование строк (5 из 6)

📶 Схема спецификации:

⦿ [[fill] align] [sign] [#] [0] [width] [,] [.precision] [type]

📶 Возможные значения:

- ⦿ fill – любой символ
- ⦿ align – "<", ">", "^", "="
- ⦿ sign – "+", "-", " "
- ⦿ width – целое число
- ⦿ precision – целое число
- ⦿ type – "b", "c", "d", "e", "E", "f", "F", "g", "G", "n", "o", "s", "x", "X", "%"

Выравнивание по
центру

`print("{:*^30.2f}".format(12.345))`

Вывод: *****12.35*****

Форматирование строк (5 из 6)

📡 Схема спецификации:

⦿ [[fill] align] [sign] [#] [0] [width] [,] [.precision] [type]

📡 Возможные значения:

- ⦿ fill – любой символ
- ⦿ align – "<", ">", "^", "="
- ⦿ sign – "+", "-", " "
- ⦿ width – целое число
- ⦿ precision – целое число
- ⦿ type – "b", "c", "d", "e", "E", "f", "F", "g", "G", "n", "o", "s", "x", "X", "%"

Ширина поля
замены

`print("{:*^30.2f}".format(12.345))`

Вывод: *****12.35*****

Форматирование строк (5 из 6)

📡 Схема спецификации:

⦿ [[fill] align] [sign] [#] [0] [width] [,] [.precision] [type]

📡 Возможные значения:

- ⦿ fill – любой символ
- ⦿ align – "<", ">", "^", "="
- ⦿ sign – "+", "-", " "
- ⦿ width – целое число
- ⦿ precision – целое число
- ⦿ type – "b", "c", "d", "e", "E", "f", "F", "g", "G", "n", "o", "s", "x", "X", "%"

2 знака в
дробной части

```
print("{:*^30.2f}".format(12.345))
```

Вывод: *****12.35*****

Форматирование строк (5 из 6)

📡 Схема спецификации:

⦿ [[fill] align] [sign] [#] [0] [width] [,] [.precision] [type]

📡 Возможные значения:

- ⦿ fill – любой символ
- ⦿ align – "<", ">", "^", "="
- ⦿ sign – "+", "-", " "
- ⦿ width – целое число
- ⦿ precision – целое число
- ⦿ type – "b", "c", "d", "e", "E", "f", "F", "g", "G", "n", "o", "s", "x", "X", "%"

Вещественное
число

```
print("{:*^30.2f}".format(12.345))
```

Вывод: *****12.35*****

Форматирование строк (6 из 6)

- 📡 f-строки – более удобная альтернатива методу `.format()`
- 📡 Для подстановки значения в строку необходимо:
 1. Поставить символ “f” перед строкой
 2. В скобках `{}` указать подставляемое значение
- 📡 Форматирование подставляемого значения осуществляется так же, как и при использовании метода `.format()`
- 📡 Данный тип форматированию доступен с версии языка 3.6

```
print(f“{12.345:*^30.2f}”)
```

Вывод: *****12.35*****

Регулярные выражения

- 📡 Регулярные выражения – набор инструментов для работы со строками
 - 🕒 Шаблоны строк
 - 🕒 Функции
- 📡 В Python регулярные выражения представлены модулем `re`
 - 🕒 Данный модуль встроен, его нужно только импортировать
- 📡 Применение регулярных выражений
 - 🕒 Проверка заполненности полей форм
 - 🕒 Обработка текстов
 - 🕒 Анализ данных

Шаблоны (1 из 2)

- 📡 Шаблон – ключевой объект в регулярных выражениях
 - 🕒 Задаётся строкой, составленной по определенным правилам
 - 🕒 Все функции регулярных выражений используют шаблоны
- 📡 Варианты использования шаблона
 - 🕒 Компиляция шаблона в специальный объект -> использование методов
 - 🕒 Использование методов модуля `re` напрямую
- 📡 Для компиляции шаблона используется метод `.compile(pattern)`

```
import re

regex = re.compile('\d')
print(regex.findall('1 2 3 44 55 6'))

print(re.findall('\d', '1 2 3 44 55 6'))
```

Шаблоны (2 из 2)

- Любая строка без специальных символов `.^$*+?{ }[]\|()` является шаблоном, которому соответствует только она сама
 - Шаблону "Hello world" соответствует только строка "Hello world"
- Каждый символ из `.^$*+?{ }[]\|()` имеет свою функцию
 - Для экранирования используется символ `\`
- Шаблон создаётся сочетанием управляющих конструкций (специальных символов) и стандартных простейших шаблонов
- Примеры шаблонов
 - `\d` – простейший шаблон (одна цифра)
 - `\w` – простейший шаблон (любая буква, цифра и символ `_`)
 - `\d\w+` – сложный шаблон (одна цифра и одна или более букв)
 - `\w{5}` – сложный шаблон (ровно 5 букв)

Базовые шаблоны (1 из 6)

Базовые шаблоны:

- ⦿ \d – одна цифра
- ⦿ \D – один символ кроме цифры
- ⦿ \w – одна буква, цифра или символ _
- ⦿ \W – один символ кроме буквы, цифры или символа _

Примеры:

Шаблон	Строки	Пример	Совпадения
\d	АИ-95	\d\d	АИ- <u>95</u>
\D	8-800-555-55-55	\D	8-800-555-55-55
\w	!Hi_5?	\w\w\w\w	! <u>Hi_5</u> ?
\W	!Hi_5?	\W	! <u>Hi_5</u> ?

Базовые шаблоны (2 из 6)

Базовые шаблоны:

- ⦿ \s – один пробел, символ табуляции или перенос строки
- ⦿ \S – один не пробельный символ
- ⦿ \b – граница слова
- ⦿ \B – не граница слова

Примеры:

Шаблон	Строки	Пример	Совпадения
\s	"\tHello world\n"	\s	"\tHello_world\n"
\S	"\tHello world\n"	\S\S\S\S\S	"\tHello world\n"
\b	Hello world	o\b	Hello <u>o</u> world
\B	Hello world	o\B	Hello wo <u>r</u> ld

Базовые шаблоны (3 из 6)

Управляющие конструкции:

- ⦿ [...] – любой из символов в скобках
- ⦿ [^...] – любой символ кроме перечисленных
- ⦿ [.-] – любой символ из диапазона

Доступные диапазоны: a-z, A-Z, a-я, A-Я, 0-9

- ⦿ Диапазоны можно заменять базовыми шаблонами

Примеры:

Шаблон	Строки	Пример	Совпадения
[..]	12:00, 12.00	\d\d[:]\d\d	" <u>12:00</u> ", "12.00"
[^..]	12:00, 12.00	\d\d[^:]\d\d	"12:00", " <u>12.00</u> "
[.-]	Hello world	[a-z]	<u>Hello</u> world

Базовые шаблоны (4 из 6)

 Указание количества повторений:

- ⦿ $\{n\}$ – ровно n повторений
- ⦿ $\{n, m\}$ – от n до m повторений
- ⦿ $\{n, \}$ – не менее, чем n повторений
- ⦿ $\{, m\}$ – не более, чем m повторений

 Примеры:

Шаблон	Строки	Пример	Совпадения
$\{n\}$	1, 12, 123, 1234	$\backslash d\{3\}$	1, 12, <u>123</u> , 1234
$\{n, m\}$	1, 12, 123, 1234	$\backslash d\{1,3\}$	<u>1</u> , <u>12</u> , <u>123</u> , 1234
$\{n, \}$	1, 12, 123, 1234	$\backslash d\{2, \}$	1, <u>12</u> , <u>123</u> , <u>1234</u>
$\{, m\}$	1, 12, 123, 1234	$\backslash d\{, 3\}$	<u>1</u> , <u>12</u> , <u>123</u> , 1234

Базовые шаблоны (5 из 6)

 Указание количества повторений:

- ⦿ ? – ноль или одно повторений; аналог {0, 1}
- ⦿ * – ноль и более повторений; аналог {0, }
- ⦿ + – одно и более повторений; {1, }

 Примеры:

Шаблон	Строки	Пример	Совпадения
?	-10, 10	-?\d{2}	<u>-10</u> , <u>10</u>
*	1, 12, 123, 1234	\d*	<u>1</u> , <u>12</u> , <u>123</u> , <u>1234</u>
+	1, 12, 123, 1234	\d+	<u>1</u> , <u>12</u> , <u>123</u> , <u>1234</u>

Базовые шаблоны (6 из 6)

Номер автомобиля

- Шаблон: `[АВЕКМНОРСТУХ]{1}\d{3}[АВЕКМНОРСТУХ]{2}\d{2,3}`
- Пример: E642OA77, O777OO77

Номер телефона

- Шаблон: `8-\d{3}-\d{3}-\d{2}-\d{2}`
- Пример: 8-800-555-55-55

Электронная почта

- Шаблон: `[a-z]+\.[a-z]+@[a-z]+\.[a-z]{1}[a-z]+`
- Пример: qdts.python@mail.ru, qdts@mail.com

Методы (1 из 4)

📶 Методы регулярных выражений, возвращающие **match object**:

- 🕒 `re.search(pattern, string)` – ищет совпадения во всей строке
- 🕒 `re.match(pattern, string)` – ищет совпадения только в начале строки
- 🕒 `re.fullmatch(pattern, string)` – проверяет соответствует ли строка шаблону

📶 **Match object** – объект, содержащий информацию о соответствии шаблону

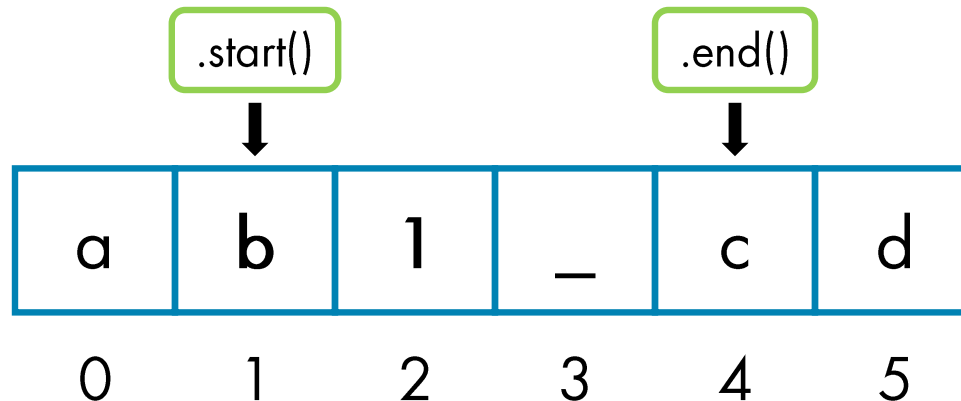
- 🕒 `.start()` – индекс в строке, с которого начинается совпадение
- 🕒 `.end()` – индекс в строке, на котором заканчивается совпадение

```
import re
print(re.match("c", "abcdef"))           None
print(re.search("c", "abcdef"))          <re.Match object; ...>
print(re.fullmatch("\w+", "ab1_cde"))    <re.Match object; ...>
```

Методы (2 из 4)

Работа методов `.start()` и `.end()`:

```
match = re.search("\w\d_", "ab1_cd")  
print(match.start())      1  
print(match.end())       4
```



Методы (3 из 4)

Методы регулярных выражений (модуль `re`):

- ⦿ `re.findall(pattern, string)` – возвращает список строк-совпадений
- ⦿ `re.finditer(pattern, string)` – возвращает итератор по всем совпадениям; каждое совпадение представлено **match object**

```
text = 'Это строка с двумя датами: 25.11.2020 и 17.08.2019'
```

```
matches = re.findall('\d{2}\.\d{2}\.\d{4}', text)
print(matches)          ['25.11.2020', '17.08.2019']
```

```
for match in re.finditer('\d{2}\.\d{2}\.\d{4}', text):
    print('Начало совпадения:', match.start())
```

Начало совпадения: 27

Начало совпадения: 40

Методы (4 из 4)

Методы регулярных выражений (модуль `re`):

- ⦿ `re.split(pattern, string)` – разделяет строку `string` по разделителю; разделитель – подстрока, совпадающая с шаблоном `pattern`
- ⦿ `re.sub(pattern, repl, string)` – заменяет все подстроки строки `string`, совпадающие с шаблоном `pattern`, на `repl`

```
text = 'Две даты: 25.11.2020 и 17.08.2019'
```

```
splt = re.split("\s", text)
```

```
print(splt)  ['Две', 'даты:', '25.11.2020', 'и', '17.08.2019']
```

```
text = re.sub('\d{2}\.\d{2}\.\d{4}', 'дата', text)
```

```
print(text)  Две даты: дата и дата
```

Флаги

- ☎ Все перечисленные функции принимают необязательный параметр **flags**
 - ☉ Данный параметр изменяет работу функций
 - ☉ Каждый флаг – константа модуля **re**
 - ☉ Если нужно передать несколько флагов, то передается их сумма
- ☎ Доступные флаги:
 - ☉ **re.IGNORECASE** – игнорирование регистра при поиске совпадений
 - ☉ **re.ASCII** – ограничивает набор символов таблицей символов ASCII
 - ☉ **re.VERBOSE** – позволяет нагляднее создавать регулярные выражения

```
text = 'KB 2, т-34'
print(re.findall(r'[А-Яа-я]+[ -]\d+', text))
print(re.findall(r'[а-я]+[ -]\d+', text, flags=re.IGNORECASE))
['KB 2', 'т-34']
```

Сравнение с несколькими шаблонами

- Символ `|` позволяет организовать сравнение строки с несколькими шаблонами
 - Строка будет совпадать с общим шаблоном, если она совпадает хотя бы с одним из шаблонов
- Пример – проверить является ли строка автомобильным номером (такси или частного автомобиля)

```
private = '[АВЕКМНОРСТУХ]{1}\d{3}[АВЕКМНОРСТУХ]{2}\d{2,3}'
taxi = '[АВЕКМНОРСТУХ]{2}\d{5,6}'
pattern = private+'|'+taxi

print(re.findall(pattern, 'e642oa777', flags=re.IGNORECASE))
['e642oa777']
```

Группы (1 из 6)

- 📡 Скобки (?:...) предназначены для группировки повторяющихся частей шаблона
 - 🕒 Для таких групп можно указывать количество повторений
- 📡 Пример – составить шаблон для IP-адресов
 - 🕒 Будем считать, что IP-адрес – это 4 числа от 0 до 999 разделенные точкой

```
# ip_pattern = '\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}'  
  
ip_pattern = '(?:\d{1,3}\.){3}\d{1,3}'  
  
print(re.findall(ip_pattern, '255.255.64.0'))  
['255.255.64.0']
```

Группы (2 из 6)

- Группы, образованные скобками (...) имеют другую функцию
- Такие группы позволяют находить внутри шаблона дополнительные совпадения
- Пример: расписание рейсов.
 - Задача: найти все рейсы.
 - Внутри каждого рейса выделить название и время отправления.

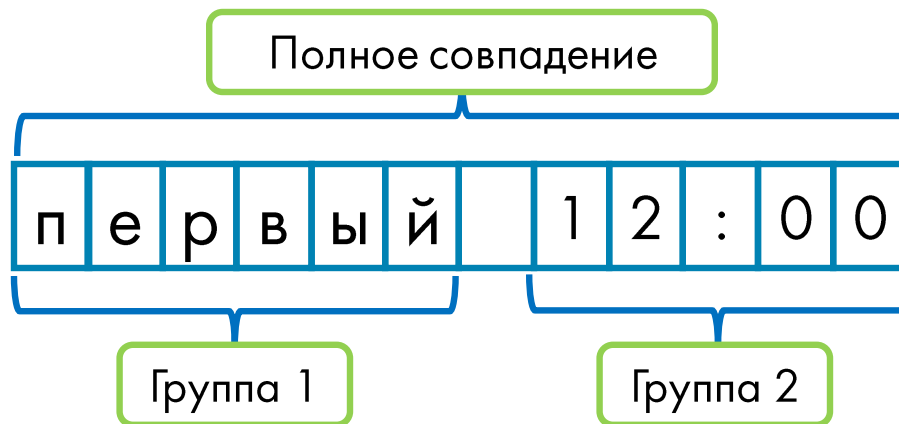
```
pattern = '[А-Яа-я]+\s\d+:\d{2}'
schedule = 'Первый 12:00\nВторой 7:30\nТретий 0:30'
matches = re.findall(pattern, schedule)
print(matches)      ['Первый 12:00', 'Второй 7:30', 'Третий 0:30']
print(
    list(map(lambda x: tuple(x.split(' ')), matches))
)  [('Первый', '12:00'), ('Второй', '7:30'), ('Третий', '0:30')]
```

Группы (3 из 6)

📶 Альтернативное решение с использованием групп

- ⦿ Необходимо в скобки заключить то, что нужно выделить отдельно

```
pattern = '([А-Яа-я]+\s(\d+:\d{2}))'  
schedule = 'Первый 12:00\nВторой 7:30\nТретий 0:30'  
matches = re.findall(pattern, schedule)  
  
print(matches)  
[('Первый', '12:00'), ('Второй', '7:30'), ('Третий', '0:30')]
```



Группы (4 из 6)

🌀 Если в шаблоне используются группы, то будут полезны некоторые методы объекта **match object**

- 🕒 `match.group(n)` – возвращает совпадение с группой `n`
- 🕒 `match.groups()` – возвращает кортеж из совпадений со всеми группами

```
pattern = '([А-Яа-я]+\s(\d+:\d{2}))'  
schedule = 'Первый 12:00\nВторой 7:30\nТретий 0:30'  
match = re.search(pattern, schedule)
```

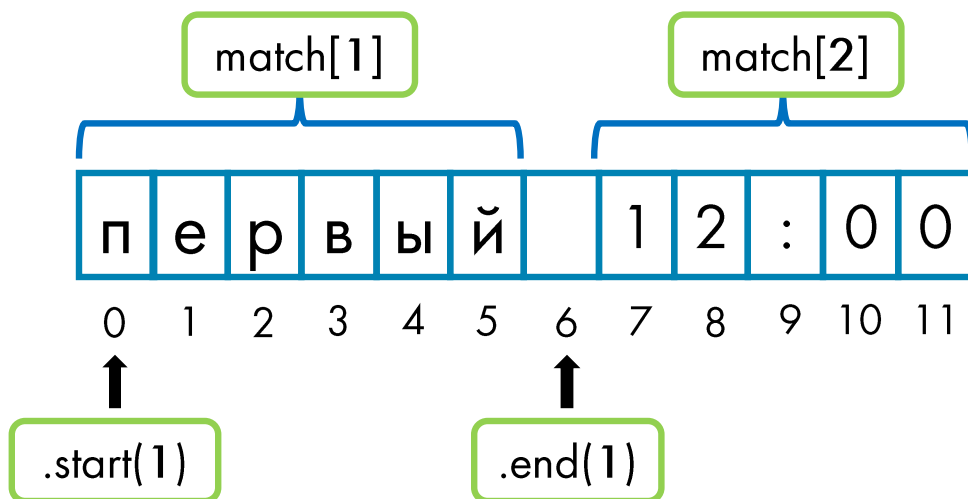
<code>print(match.group(0))</code>	Первый 12:00
<code>print(match[1])</code>	Первый
<code>print(match.group(1, 2))</code>	('Первый', '12:00')
<code>print(match.groups())</code>	('Первый', '12:00')

Группы (5 из 6)

☞ Для каждой группы доступные индексы начала и конца совпадения

- ☉ В методы `.start()` и `.end()` передаётся дополнительный аргумент – номер группы

```
print(match.start(1))    0
print(match.end(1))     6
```



Группы (6 из 6)

- При использовании групп работа некоторых методов меняется, а другие методы получают дополнительные возможности
 - `re.findall()` начинает выводить список кортежей групп
 - Метод замены `re.sub()` становится более гибким

```
pattern = '([А-Яа-я]+\s(\d+:\d{2}))'  
schedule = 'Первый 12:00\nВторой 7:30\nТретий 0:30'  
  
print(re.sub(pattern, r'\2 - \1', schedule))  
12:00 - Первый\n7:30 - Второй\n0:30 - Третий
```

К совпадениям с группами можно получить доступ через символ `\` и номер группы

Итоги

В данной лекции были рассмотрены следующие темы:

-  Строки в Python
-  Запись строк
-  Специальные символы
-  Простейшие операции
-  Методы строк
-  Встроенные строковые функции
-  Форматирование строк
-  Регулярные выражения
-  Шаблоны
 -  Базовые шаблоны
 -  Сложные шаблоны
-  Методы регулярных выражений
-  Группы